Slide 1

Git, GitHub, and VS Code

Agentic AI for Research: From One Task to Orchestration

Tomas Larroucau

Arizona State University

Monday, October 26, 2026

Slide 2

Why This Workshop?

The Problem:

- Research projects grow messy fast
- Code, writing, and decisions drift apart
- More agents can mean more work to supervise

What We Will Do:

- Work in one research environment
- Turn a task into a reviewable change
- Coordinate agents with clear ownership

By the end

You can give an agent a bounded task, inspect its changes, and decide whether the work is ready to share.

Slide 3

Our 90 Minutes

Minutes	What we do
0–5	The research workflow and today's goal
5–12	VS Code and the research project
12–22	Git, issues, and reviewable changes
22–32	Claude Code and Codex: practical comparisons
32–50	AI-assisted writing: claims, exhibits, author direction
50–75	Revision demo: comments, issues, agents, draft
75–84	Orchestration: ownership, sessions, resources
84–90	Questions and one next step

Preparation decision still open

The revision case is being selected. Use clearly labeled prepared checkpoints and keep scientific decisions with the author.

Slide 4

The Symbiosis: Agents as Builders



- **VS Code**: read, write, run, and review in the project
- **Git and GitHub**: saved history and shared decisions
- **Agents**: help build and check the workflow itself

The dinosaur can help build the fence. We still inspect the fence.

Git can restore recorded files. It cannot retract a message, recover every unsaved file, or undo an external action.

Slide 5

VS Code: An Integrated Research Environment

Read and Write

- Browse papers, notes, and project files
- Edit LaTeX and Markdown
- Preview a PDF beside its source
- Search the whole project

Run and Collaborate

- Run scripts in the terminal
- Inspect Git diffs and history
- Work with issues and pull requests
- Use agent extensions or CLI sessions

Demo: Open the project folder. Read the README, preview the slides, inspect a diff, and open an agent session.

PDF, language, and agent features depend on the extensions you install. Start with what your project needs.

Slide 6

Reproducible Research Template

- `data/raw/`: teaching data, kept unchanged
- `scripts/`: processing and analysis
- `output/`: generated figures and tables
- `tex/`: paper and Beamer sources
- `Makefile`: build steps
- `README.md`: how to use the project
- `AGENTS.md`: project guidance
- `.gitignore`: untracked files to exclude

Key Principle: Generate research outputs from scripts. Review source edits and record the decisions behind them.

Template: `https://github.com/tlarroucau/AI_workshop`

Build commands in the appendix

Slide 7

Workshop Prerequisites

Follow and Review

- A browser and the workshop materials
- VS Code and Git for the local exercise
- A local copy of the template
- A partner if setup is incomplete

Run an Agent Yourself

- One supported agent, signed in
- Check your account's access and limits
- GitHub account for hosted collaboration
- Python and LaTeX only for full builds

Today's exercise

The writing review needs only the supplied passages. The scientific demo uses prepared inputs and outputs. Install Python and LaTeX only if you plan to run its full pipeline.

Slide 8

Git Core Concepts

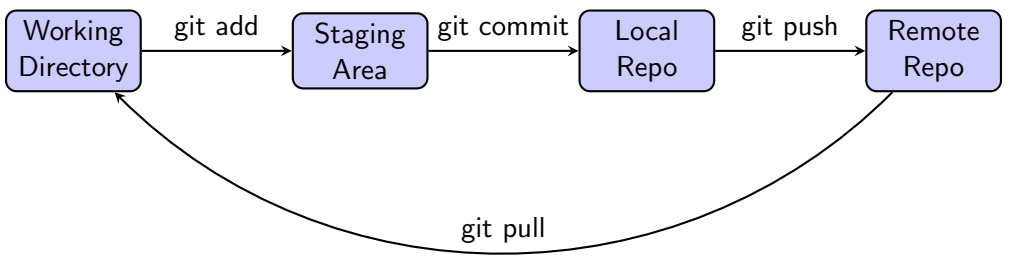
1. **Repository (repo)**: Project folder tracked by Git
2. **Commit**: Snapshot of the changes you staged
3. **Branch**: Parallel version of your repository
4. **Merge**: Combine changes from different branches
5. **Remote**: Another repository (often hosted on GitHub)

Basic Workflow

```
git add file.py          # Stage changes
git commit -m "message"  # Save snapshot
git push                 # Upload to remote
```


Slide 9

The Git Workflow



- Edit files in working directory
- Stage changes you want to commit
- Commit records the staged changes in local history
- Push shares with collaborators

Slide 10

GitHub Issues: Shared Research Context

An issue records the task

- Research goal and why it matters
- Relevant files and decisions
- Owner and allowed scope
- Evidence required to finish

Example: specify a research exhibit

Define the population, comparison, units, uncertainty, and output asset. Link the approved model and input version. Assign a writer and an independent checker.

Coauthors can read the issue after the chat ends. Ask each new session to read it and report what remains unresolved.

The example is a task brief. Creating or updating a hosted issue requires the appropriate access and authorization.

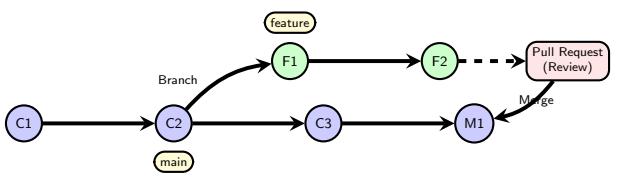
Slide 11

GitHub Workflow: Pull Requests

What are Pull Requests (PRs)?

- Propose changes to repository
- Enable code review before merging
- Discuss implementation details

PR Workflow:

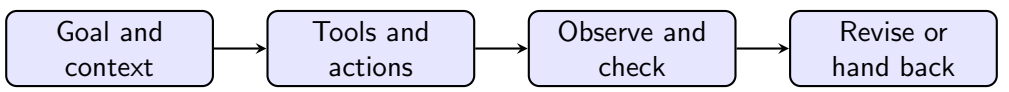


1. Create and push feature branch
2. Open Pull Request on GitHub
3. Review, discuss, and fix
4. Merge when approved

Review: Does the diff satisfy the issue? What evidence supports merging?

Slide 12

What Is Agentic AI?



- An agent can read files, edit, run commands, and use connected tools when permitted.
- The model, its tools, context, and permissions jointly determine what it can do.
- Completion is a claim to check against the requested output and evidence.

Model, Agent, and Interface

Three different choices

Model: the system generating responses.

Agent harness: instructions, tools, permissions, and execution loop.

Interface: terminal, editor, desktop app, or web session.

Choosing a Claude model inside Copilot still uses Copilot's agent harness. It does not turn that session into Claude Code.

Compare workflows on the same task: context used, control of edits, review quality, time, and allowance consumed.

VS Code: agent harnesses

Claude Code and Codex: Context and Planning

Feature / use	Claude Code	Codex
1. Instructions Project conventions	`CLAUDE.md`; can import `@AGENTS.md`. Recent versions also support an `AGENTS.md` fallback.	`AGENTS.md`, including guidance scoped to subdirectories.
2. Skills Repeat a workflow	`SKILL.md` in `.claude/skills/`; invoke the named skill.	`SKILL.md` in `.agents/skills/`; use `/skills` or `$name`.
3. Plan first Discuss before editing	`--permission-mode plan` or Shift+Tab.	`/plan` in supported sessions; read-only permissions add an explicit edit boundary.

Research use: store project conventions once; make a table-audit skill; agree the scope of a revision before editing.

Claude instructions, skills, planning; Codex instructions, skills, planning.

Claude Code and Codex: Output, Tools, Control

Feature / use	Claude Code	Codex
4. Output style Match the audience	Named styles; try `/output-style explanatory` while learning.	Instructions and `/personality` where supported; these are not identical to Claude's style system.
5. Tools / MCP Connect the workflow	Built-in tools plus configured MCP servers.	Built-in tools plus configured MCP servers.
6. Permissions Bound the actions	Permission modes and shell sandboxing are separate controls.	Approval policy and sandbox settings are separate controls.

Research use: request a short instructor explanation, connect only needed tools, and inspect what can read, edit, execute, or send data.

Claude styles, MCP, permissions; Codex commands, MCP, security.

Claude Code and Codex: Sessions and Remote Work

Feature / use	Claude Code	Codex
7. Sessions Resume a task	`--continue`, `--resume`, or `/resume`.	`codex resume`; `/compact` manages long context. Naming and forks depend on the surface.
8. Remote control Supervise work elsewhere	Remote Control connects to a session on a running host. Eligibility and admin settings apply.	Remote connections are surface-dependent. CLI remote control is experimental; check the installed version.

Save research decisions in files or issues. A resumed chat is useful context, but collaborators need a durable record.

Remote control is a view into work on another host, not automatic migration to cloud execution. Rehearse this optional demo in advance.

Claude sessions, remote control; Codex commands, remote connections.

Claude Code and Codex: Delegation and Review

Feature / use	Claude Code	Codex
9. Delegation Separate bounded tasks	Subagents use separate contexts. `--worktree NAME` isolates a working directory.	Subagents and Git worktree workflows are supported; setup depends on the surface.
10. Review Challenge the change	Ask a separate session or reviewer agent to inspect the task, diff, and checks.	`/review`; connected GitHub review is a separate integration.

Research use: one agent drafts, another checks sources, and a reviewer looks for mistakes. Assign each writer distinct files or a worktree.

Claude's experimental `/advisor` consults another model; it is different from Plan Mode. Availability and extra usage depend on provider and billing.

Claude subagents, advisor; Codex subagents, worktrees, review.

AI-Assisted Writing Starts with the Argument

1. **Claims:** What do we know, and what evidence supports it?
2. **Interpretation:** What economic mechanism does the evidence establish?
3. **Exhibits:** Specify the figure or table that lets a reader assess the claim.
4. **Prose:** Write the paragraph around the verified evidence.

A useful first request

Read the approved results and the surrounding section. List the claims each exhibit supports, the qualifications it needs, and any missing evidence. Then propose the paragraph's argument.

Keep missing results explicit. A fluent paragraph cannot supply an estimate, citation, or identification argument.

Slide 19

Four Layers of Writing Direction

Layer	What it controls in this task
Task prompt	Revise these two sentences using the approved claims. Preserve their meaning and return a diff.
General instructions	Reader, author voice, notation, evidence rules. Put persistent project guidance in `AGENTS.md` / `CLAUDE.md`.
Output style	How the agent responds. Claude: `/output-style explanatory` for teaching. Specify manuscript voice separately.
Reusable skill	Read neighboring prose, trace claims to evidence, edit, check notation, render the exhibit, and review.

An output-style setting cannot establish the argument or verify a claim.

Local author skills; Claude output styles; Codex skills. Codex instructions / personality are not an equivalent output-style system.

Slide 20

Author Instructions: Write for the Paper's Reader

- Use established terms; do not coin labels.
- Remove stock trailing clauses and contrastive padding.
- State the economic argument directly.
- Keep chat history and internal workflow out of the manuscript.
- Introduce notation only when necessary; define every new symbol.

Skills I use

`econ-write`: organize the economic argument.

`academic-draft-editing`: match the draft, check provenance, inspect exhibits.

`humanizer`: remove filler while preserving meaning and voice.

For questioning: Matt Pocock's `grill-me`; `grill-with-docs` adds repository glossary and decision records. Details and install commands follow in the reference slides.

Teaching summary of Tomas's user-owned research skills and author instructions. These are preferences for this draft, not a universal ban on technical language or useful contrasts.

Slide 21

A Synthetic Paragraph: Give the Agent the Claims

Approved content: Students pay an application cost before the offer is known. The expected value of enrollment and continuation enters their application decision. No estimated magnitude is available.

Before: plausible wording, poor direction

Our forward-looking access-persistence nexus leverages a dynamic framework, underscoring the pivotal role of expectations. This is not merely about admission but about sustained success. After resolving the implementation issues, we can now show substantial gains.

Author-directed revision

Students pay an application cost before learning whether they receive an offer. Their application decision depends on the expected value of enrolling and continuing in the program.

Direction: Use only the approved content. Keep the timing and mechanism. Remove coined labels, workflow history, and unsupported results. Introduce no notation.

Synthetic teaching example written for this workshop; no empirical finding or selected research-demo model is implied.

Slide 22

A Real Draft Placeholder Specifies the Exhibit

CuposBEA draft: selected sentences from a figure specification

Typeset excerpt

Both vertical axes are in rank points of the main outcome, with horizontal zero lines.

Use the same students, prediction convention, kernel, and bandwidth across panels, with student weights within each track.

Draw each track over its adequately populated support; suppress windows with fewer than 20 students and report the track sample sizes and support.

What this gives the agent: units, comparability across panels, weighting, and an explicit support rule.

The full specification also requests uncertainty calculations and an output asset. Those details require the chosen estimator and data.

Actual author-requested excerpt, typeset for teaching. Surrounding draft, results, model notation, and private asset path omitted. The 20-student rule is this exhibit's specification, not a general statistical recommendation.

Slide 23

A Precise Figure and Table Request

Synthetic teaching brief; outputs have not been computed.

Population	Same synthetic student cohort under both fees. Include nonapplicants in the denominator.
Figure	Two panels: enrollment and persistence probabilities for all students. Same five preparation groups on the horizontal axis; baseline and policy means in each.
Table	For each outcome and group: count, both means, difference in percentage points, and interval. Use the figure's underlying output.
Validation	Check denominators, units, groups, and missing values. Name the uncertainty method; separate Monte Carlo error when relevant.
Delivery	Generator, data dictionary, PDF figure, LaTeX table, caption, and draft diff. Mark missing inputs; never invent values.

Before execution: agree on the fee change, persistence horizon, estimator, and uncertainty procedure.

Slide 24

The Author Still Reviews the Writing

- Read the passage as someone who has only the paper.
- Check every claim against the source, exhibit, and intended interpretation.
- Check whether the edit changes the argument or adds a new qualification.
- Recheck the author rules: a model can drift back to default phrasing.

Experience and evidence

Tomas's preference for OpenAI models for some writing tasks reflects his experience. Better coding performance does not establish better research writing.

Discussion: Which choices should the agent propose, and which must remain the author's? Check the actual venue's disclosure rules; the author remains responsible for the final text.

Slide 25

Revision Demo: From Comments to a Verified Draft

Revision case pending: CuposBEA or the dropout-matching JMP; use a permitted Refine report or a clearly labeled synthetic comment set.

1. Triage comments into scientific decisions, evidence requests, and writing edits.
2. Draft issues by argument, files, and dependencies; assign visible task owners.
3. Parallelize independent work; sequence coupled math, code, and draft edits.
4. Review checks and exhibits, then inspect proposed PRs and integration.
5. Verify the revised argument and compiled draft against the original comments.

Rehearse the handoffs

Prepare the baseline, small checks, and fallback checkpoints. Say which outputs were prepared and which were produced live.

Slide 26

A Research Prompt with Clear Ownership

Researcher: choose the question and design

Read the model, code, draft, and approved task brief. Identify the assumptions the extension changes and the claims we could support. List decisions I must make before implementation.

Agents: work against the same agreed specification

Theory agent: derive and explain. Implementation agent: edit the assigned code in an isolated worktree. Independent checker: construct the small-case reference calculation and challenge assumptions. Draft integrator: use only verified exhibits and approved interpretation.

Specify inputs, output files, checks, budget, and stop conditions for each role. Ask for mathematical disagreement before accepting a polished draft.

Slide 27

A 25-Minute Demo with Visible Checkpoints

Minutes	What participants inspect
0–4	Comments, author decisions, and revision scope
4–12	Issue drafts, dependency map, and distinct task owners
12–18	Parallel work, scientific disagreement, and checks
18–23	Exhibits, candidate PRs, and integration review
23–25	LaTeX diff and compiled manuscript

Participant role: identify one claim and the evidence needed to accept it.

This is the prepared demonstration plan. The case and its checkpoints are not yet selected. If execution stalls, inspect a labeled prepared checkpoint.

Slide 28

You Choose the Level of Engagement

You specify	What the agent can help produce
Research question	Alternatives, assumptions, evidence to collect
Project design	Task breakdown, dependencies, risks, acceptance criteria
Bounded task	A proposed change and evidence that it works
Exact implementation	Specific edits, commands, or code you want executed

Move between levels as needed. Keep ownership of research choices and the decision to accept or share results.

Slide 29

Parallel Work Needs Distinct Ownership

Session	Task and output	Scope	Status
Planner	Agree scope and task brief	Read-only	Ready
Claude Code A	Derivation and assumptions	Theory; worktree A	Working
Codex B	Implement agreed model	Code; worktree B	Working
Reviewer C	Independent reference check	Separate check; findings	Working

Each task has one owner, a defined input, an expected output, and a stop condition. The labels here are an example, not a model ranking.

Control: Inspect progress, interrupt, narrow scope, or reassign. Parallelize independent work; review follows a candidate change.

Slide 30

One Machine: Clearly Labeled Sessions

A simple setup

- Open separate terminal or editor sessions
- Label provider, task, and working folder
- Keep one shared task brief or board
- Give each writer an isolated worktree

Optional shared view

- VS Code documents discovery of local external Claude Code and Codex sessions
- Enable the sessions list's *External* filter
- Check provider support before continuing a session

Independent labeled sessions are enough. Local session discovery does not establish discovery of remote SSH or tmux sessions.

VS Code: manage sessions. Codex continuation has a single-writer requirement; see appendix.

The Orchestrator Keeps the Big Picture

Personal assistant / orchestrator

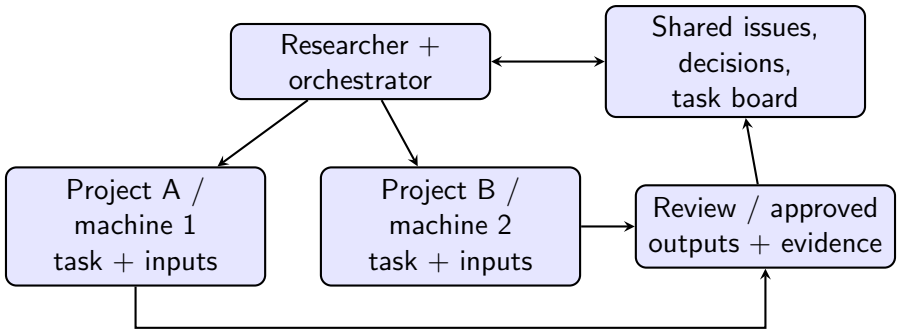
- Research goals and project priorities
- Decisions, dependencies, and open questions
- Task board and resource allocation
- Reports back in terms of outcomes

Implementation sessions

- A bounded task and local instructions
- Relevant files and tools
- Changes, checks, and evidence
- A concise handoff when done or blocked

Record the shared decisions outside an implementation-heavy terminal chat. Give collaborators the issue, candidate diff, and verification evidence.

Several Projects and Machines



Command Center demo: a private illustration of this workflow. This diagram uses generic projects and machines.

Track input versions, execution location, allowance, and output destination. Share only the artifacts and evidence collaborators need.

Next Steps and Questions

1. Choose one bounded scientific or writing task in your project.
2. Write its scope and acceptance criteria.
3. Use one agent, inspect the diff, and verify the result.
4. Add parallel sessions when the tasks can be separated clearly.

Discussion: Which part of your workflow would benefit most? What would you need to check before trusting the result?

Template: `https://github.com/tlarroucau/AI_workshop`
Build commands, worktree examples, and official documentation follow.

Matt Pocock's Skills: Ask Better Questions

`grill-me`

- An interview for an idea or plan; useful beyond code.
- Uses `grilling` to ask dependent questions and recommend answers.
- Normally works without writing files.

`grill-with-docs`

- Reads the codebase and records agreed domain knowledge.
- Uses `grilling` and `domain-modeling`.
- Maintains a glossary and qualifying architecture decision records.

Our research adaptation

Interview me about the claim, reader, mechanism, evidence, and exhibit before drafting. Investigate facts available in the project; ask me to resolve scientific decisions.

This is an adaptation for research writing. `grill-with-docs` is not a generic paper-interrogation skill.

Matt Pocock, grill-me, grill-with-docs; MIT, 2026. See source notes for dependencies.

Install the Skill Collection Before the Session

Claude Code plugin route

```
claude plugin install mattpocock-skills@claude-plugins-official
```

Codex plugin route

```
codex plugin marketplace add mattpocock/skills
codex plugin add mattpocock-skills@mattpocock
```

Alternative: select editable skills interactively

```
npx skills@latest add mattpocock/skills
```

- Choose a plugin or copied skills per agent; avoid duplicates.
- Preserve dependencies. Copying one `SKILL.md` is insufficient.
- `grill-me` needs no repository setup. The full engineering workflow has `/setup-matt-pocock-skills`.

Commands documented in Matt Pocock's README, checked Oct. 25, 2026. Availability depends on client/version.

These are preparation instructions; no package was installed for this revision.

Install and Invoke Your Own Research Skill

Inspect the full skill folder first: instructions, scripts, dependencies, and the files it may change.

	Repository location	Invoke in the session
Claude Code	`.claude/skills/research-edit/SKILL.md`	`/research-edit`
Codex	`.agents/skills/research-edit/SKILL.md`	`$research-edit` or `/skills`

Minimal header; replace the body with the reviewed procedure

```
---
name: research-edit
description: Edit a research paragraph using approved claims.
---
Read the source passage and author-approved evidence first.
```

Keep required supporting files beside the skill. Start a fresh session and check discovery. User-level locations are `~/.claude/skills/` and `~/.agents/skills/`; cloud surfaces differ.

Claude skills; Codex skills. Guidance for local sessions; inspect current provider documentation before classroom setup.

Install One Agent, Then Verify It

Official terminal installers for macOS / Linux

```
# Claude Code
curl -fsSL https://claude.ai/install.sh | bash
claude --version

# Codex
curl -fsSL https://chatgpt.com/codex/install.sh | sh
codex --version
```

1. Open the teaching project, start `claude` or `codex`, and sign in.
2. Check the working folder, account access, permissions, and available skills.
3. Ask for a read-only project explanation before permitting an edit.

Use the official instructions for your OS. Windows users should follow the linked Windows route.

Claude Code Quickstart; Codex CLI. Verified Oct. 25, 2026. Install and authenticate before the workshop; both subscriptions are not required.

Separate Worktrees for Separate Writers

Branch

- A line of Git history
- Switching branches changes the current checkout

Worktree

- A separate working directory
- Lets writers use different branches at once

Coordination still matters

Agree which files each agent owns. Keep outputs separate, review overlapping edits, and integrate one change at a time. Worktrees isolate files; they do not isolate shared databases or external services.

Commands are in the appendix. Start from an agreed commit and preserve existing work before branching.

Allocate Models and Attention by Task

- Use a lower-cost available option for a bounded, easy-to-check task.
- Reserve scarce allowance and deeper reasoning for hard design choices, ambiguous failures, and consequential review.
- Stop duplicate work. Reuse a concise task brief and record useful results.
- Give reviewers the task, diff, and evidence. Ask them to find mistakes independently.

Keep the evidence local to the claim

"It worked on this task" is an observation. "This model is always best" requires evidence we do not have here.

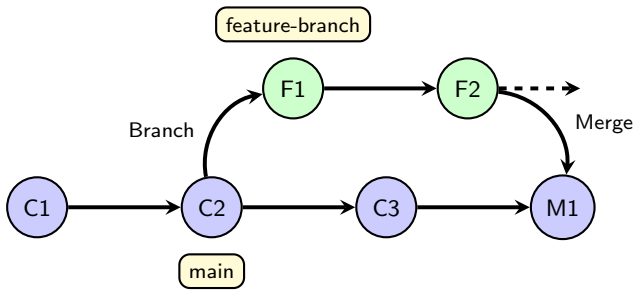
Finish the Task, Then Verify Closeout

1. Compare the result with the issue's acceptance criteria.
2. Inspect the diff, checks, and rendered outputs where relevant.
3. Resolve reviewer findings or record what remains open.
4. When sharing is authorized, link the PR to the issue and obtain review.
5. After integration, verify the intended result and the issue's status.

A useful handoff

What changed; where the output is; what was checked; what is blocked; which decision is needed next.

Visualizing Branches



- **Branch**: Create a parallel timeline (`git checkout -b`)
- **Commit**: Work on the feature branch (F1, F2)
- **Merge**: Combine feature history back into main (`git merge`)
- *Note: The branch remains open after merging unless deleted*

Essential Git Commands

```
git status                  # Check current state
git diff                    # Review unstaged edits
git diff --staged           # Review staged edits
git switch -c docs-quickstart  # Create a task branch
git add README.md           # Stage the intended file
git commit -m "Clarify Quick Start"
git log --oneline           # View saved history
git fetch                  # Retrieve remote history
```

Inspect local work before switching or integrating branches. Pull behavior depends on configuration. Push and PR creation share work with the remote repository.

Slide 43

Worktree Commands: One Writer per Folder

```
# From the repository, start at an agreed commit:
git status
git worktree add ../workshop-docs -b docs-quickstart HEAD
git worktree list
# Start the writing agent inside ../workshop-docs
# Give the reviewer the candidate diff and task brief.
```

- Uncommitted edits are not copied into a new worktree.
- Use distinct output paths when sessions run concurrently.
- Preserve and integrate needed work before removing a worktree.

Git: worktree

Slide 44

Git Best Practices

Commit Messages

- Be descriptive: "Clarify README build instructions"
- Keep each commit focused on one task
- Link to the relevant issue when there is one

What to Commit

- Source code, documentation, and small permitted teaching data
- Generated outputs only when the project's sharing policy calls for them
- Keep credentials and restricted data outside the repository

Slide 45

The .gitignore File

Example patterns

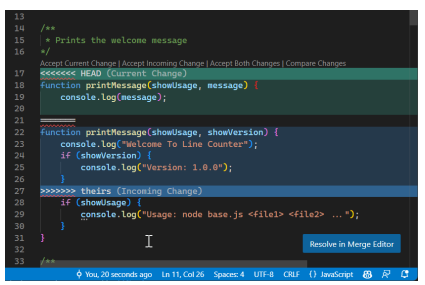
```
__pycache__/
.venv/
*.aux
*.log
output/
.env
```

- Applies to untracked files
- Does not remove a file already tracked
- Can appear in subdirectories
- Does not erase secrets from history

Inspect `git status` and the staged diff before committing. An ignore rule helps prevent accidents; it does not enforce data policy.

Slide 46

Resolving Merge Conflicts in VS Code



- VS Code shows **conflict markers**: `<<<<<<< HEAD` (yours) vs `>>>>>>> theirs`
- One-click options: *Accept Current*, *Accept Incoming*, *Accept Both*, or *Compare*
- Use **Resolve in Merge Editor** for a side-by-side 3-way diff

Slide 47

Useful VS Code Features

Navigate and Review

- Command Palette: Ctrl/Cmd+Shift+P
- Quick file open: Ctrl/Cmd+P
- Search files, symbols, and text
- Source Control: staged and unstaged diffs

Add What You Need

- LaTeX Workshop: source and PDF
- Python / Jupyter: scripts and notebooks
- GitHub Pull Requests and Issues
- Your chosen agent's extension

Open the project folder, select the correct environment, and use workspace settings for project-specific tools. Check Keyboard Shortcuts for platform differences.

Slide 48

MCP: Model Context Protocol

- MCP connects an agent application to tools and resources exposed by a server.
- A GitHub connection can expose repository operations, depending on the server and permissions.
- Native tools and command-line tools are also valid integration paths.
- Inspect the requested permissions and configure only the access the task needs.

For this workshop: the instructor can use a prepared connection. Participants do not need to configure MCP for the writing review.

MCP architecture; provider setup links in the source notes.

Slide 49

Data Privacy and Permissions

- Use public or permitted teaching material for the exercises.
- Check institutional policy and the account's terms before providing research data.
- Local execution does not by itself mean local model inference.
- Inspect connected tools, network access, and output destinations.
- Ask before actions outside the task's agreed scope, including publishing or sending messages.

An offline workflow requires checking the whole configuration. A locally installed editor or agent alone does not establish that no data leaves the machine.

Slide 50

Overleaf, Statistical Software, and VS Code

Statistical Software

Use the language extension or terminal workflow supported by your installation. Check paths, licenses, working directories, and the exact command on a small file.

Overleaf

Agree where the authoritative draft lives and who synchronizes changes. Check available Git or GitHub integration for the account. Integrate reviewed changes through one owner, then inspect the compiled PDF.

Avoid concurrent edits to the same checkout through multiple sync systems. Keep a recoverable copy before synchronizing.

Slide 51

The Makefile Approach

A recipe for the project

- Targets declare dependencies and commands
- Read a target before running it
- Inspect outputs after a successful build

In this repository, `make slides` first runs the analysis through `update-outputs`.

Commands for this revision

```
make help
make slides-only
# Full research workflow:
make all
```

`slides-only` builds the Beamer deck from the included assets. It does not rerun the analysis.

Back to Project Structure

Slide 52

Research Exercise: Audit the Robustness Check

1. Open the existing paper's robustness table and its discussion.
2. Trace the table to `scripts/analysis.py`: default OLS versus standard errors clustered by `region_id`.
3. Ask one agent to check the implementation and another to compare the table with the prose.
4. Require file/line evidence, assumptions about clustering, and a list of claims that need a rerun to verify.
5. Draft a bounded issue and review plan before changing the analysis.

The original clustered-SE task is already implemented. Audit the existing work, or choose an instructor-approved extension. This is a reference exercise; the stronger main demonstration is being selected.

Slide 53

VS Code Session Discovery: Scope Matters

- Local external Claude Code and Codex sessions can be discovered. They are hidden by default; enable *External* in the sessions-list filter.
- The setting is `chat.agentSessions.showExternal`.
- Codex continuation requires a single writer: fully quit the other application, then retry in VS Code.
- The OpenAI extension and experimental Codex Agent Host are distinct integration surfaces.

Remote SSH/tmux session discovery is not established by the local-session documentation. Rehearse the exact setup before promising it in a demo.

Manage sessions; agent harnesses.

Slide 54

Resources

Project Workflow

- Git documentation
- GitHub documentation
- VS Code documentation
- GitHub Skills

Agent Documentation

- Claude Code
- Codex
- Model Context Protocol
- Workshop repository

Product availability depends on version, surface, account, and organization settings. The accompanying source notes record the documentation used for this revision.